TYPES, VARIABLES & SEQUENCES! (LECTURE)

IMPORTANT REMINDERS

1. Recitations start this week. Check Path@Penn and/or your email to verify where you should go!
2. HW00 due on Wednesday, September 10th at 11:59. You submit this on **Gradescope**, which you should navigate to through Canvas.
    1. Don't leave submission to the last second.
    2. Look at the output that is shown to you! If your files have the wrong name or other issues, it will tell you. *You will lose points for those things!*
3. Office Hours start this week. Mon-Thurs, mostly in AGH 200. Check the website for details.

NUMERICAL TYPES

In python, we can store numbers in variables. However, there is a distinction between two types:

- `int` These are Integers, meaning any positive or negative value (or zero).
  - e.g. `0`, `-3200`, `10`, `299792458`
- `float` These can store rational numbers and some special values
  - e.g. `3.14`, `8.3144`, `1.4142`, `2.718`, `infinity`, `-infinity`

NUMERICAL OPERATORS

NAME	SYMBOL	USE
addition	+	x + y
subtraction	–	x – y
division	/	x / y
multiplication	*	x * y

OPERATOR PRECEDENCE

Basic order of operations: (First evaluated to last evaluated)

- Parentheses `()`
- exponents `**`
- multiplication/division/mod `*` `/` `//` `%`
- addition / subtaction `+` `-`
- comparisons and membership `in` `not in` `<` `>=` `==` etc.
- `not`, then `and`, then `or`

You do not need to memorize these. When in doubt, use parentheses.

MIXING NUMERICAL TYPES

- If you use an operator on two `int`s you get an `int`
  - (except for `/`, which gives a `float`. Why?) 🧠 🤔
- If you use an operator on two `float`s, the result will be a `float`
- if you operate on an `int` and a `float` you get a `float`. (Why?)

LECTURE ACTIVITY

What are the *resulting types* of the expressions and *what will be printed*?

S7:

```
x = 3 + 0.5 * 2
print(x)
```

S8:

```
x = (2 * 8) / 3
print(x)
```

MORE ASSIGNMENT OPERATORS

There are also a few more operators worth covering, like `+=`:

```
x = "h"
x += "i" # equivalent to x = x + "i"
print(x) # 🖨️ ➡️ "hi"
```

This operator "adds" the two values together and sets the variable on the left equal to the result.

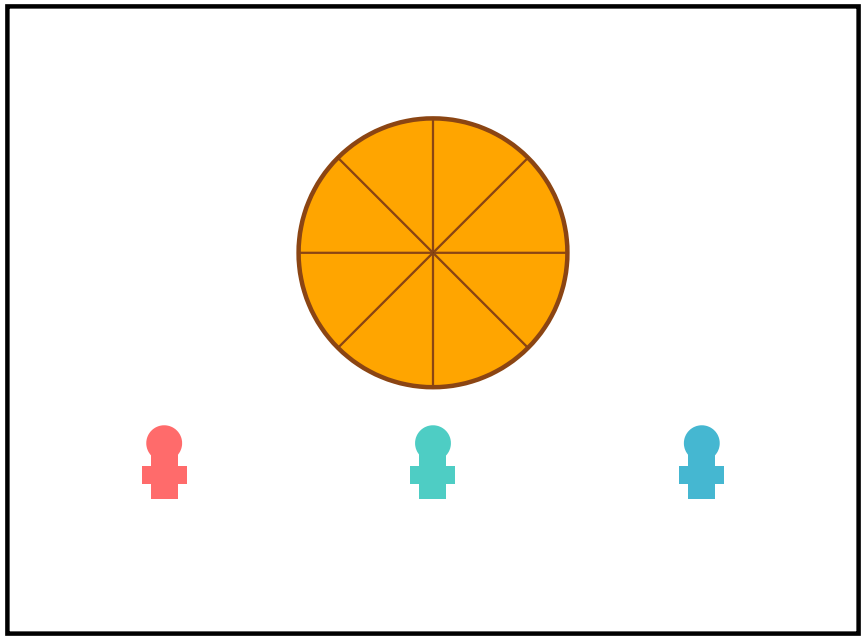
Other variants: `-=`, `*=` and `/=` exist, too!

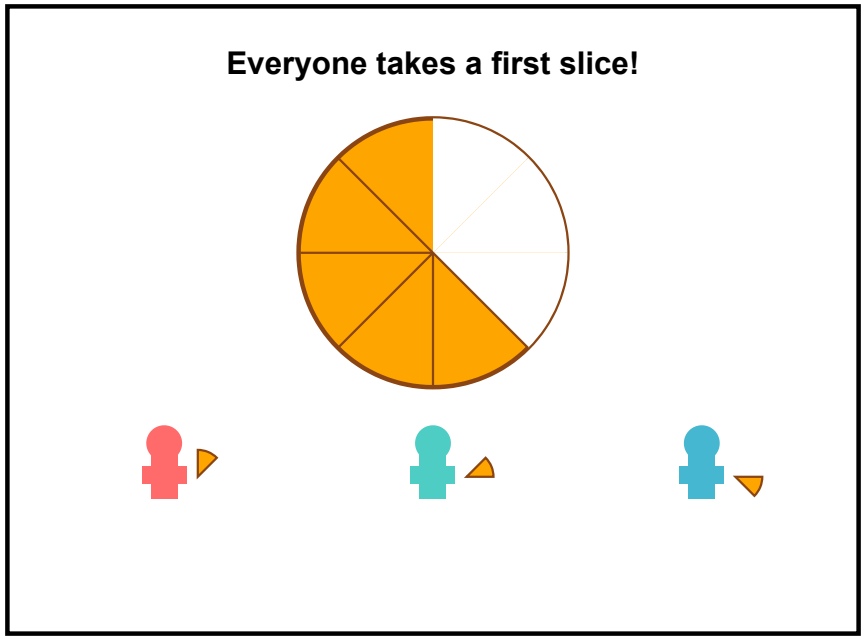
OTHER ARITHMETIC OPERATORS

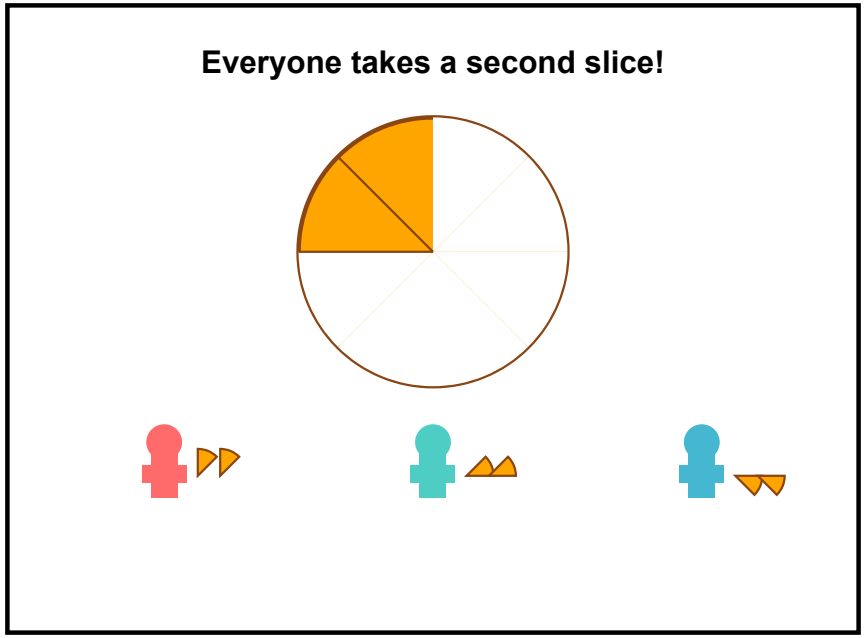
- `**` used for exponents.
  - e.g. 5 squared is written as `5 ** 2`
- `//` used for "integer division, rounds the result towards 0
  - `int // int` evaluates to an `int`
  - `3 // 2` evaluates to `1`
- `%` called "modulo" used to get the remainder of a division.
  - `5 % 2` evaluates to `1`
  - `9 % 3` evaluates to `0`

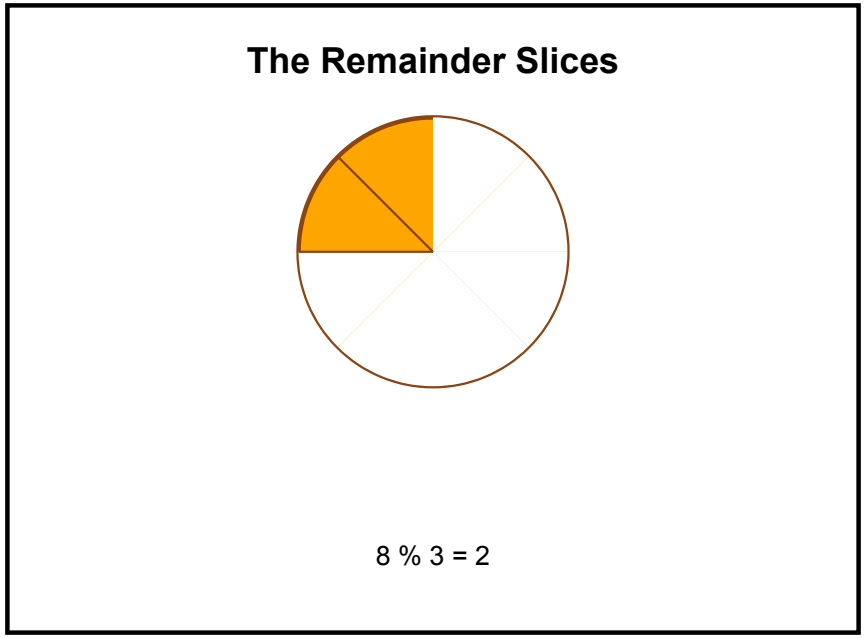
THE PIZZA ANALOGY

(I used to do a long-division demonstration in class but that's no fun)









LECTURE ACTIVITY

S9:

What does this evaluate to? `(10 % 3) ** 2 // 5`

REVIEW: BOOLEAN TYPE

Another type that exists is `bool` which can represent two values `True` or `False`

```
x = True
y = False
print(x)
```

We can use the operators `and`, `or` and `not` on booleans

What gets printed? **S10**

```
a = False
b = True
c = (not a or b) and not (a and True)
print(c) # 🖨️ ❓
```

REVIEW: COMPARING

A common way to get boolean values is through comparison.

- `==` checks if two things are equal
- `!=` checks if two things are NOT equal

Some examples:

- `"Hello" == "hello"` evaluates to `False`
- `5 != 3` evaluates to `True`
- `"hi" == "hi"` evaluates to `True`

REVIEW: ORDERING

There also exist operators to check for:

- `<=`: less than or equal to
- `>=`: greater than or equal to
- `<`: less than
- `>`: greater than

ARITHMETIC & RELATIONS

Write some code to determine if the variable x contains an even integer L11

```
x = <some_number>
is_even = ____________________
```

The `<some_number>` bit is not actual Python; we want you to think about how to make this work for a variable `x` even if you, the human, do not currently know what's inside!

TYPE CONVERSION

Allows you to convert from one type to another

```
x = int("1300")   # converted str to int
z = str(x)        # Z has str conversion of x "1300"
a = bool("True")  # a has bool value True
f = float("3.14") # f has float value 3.14
```

DISCUSS: COMPLEX REASSIGNMENT

What does `z` evaluate to after this code is run? **C12** (so you have scratch space)

```
x = 3
y = "love <"
z = str(x > 0 and x <= 6) + " " + y + str(x)
```

Compressed Order of Operations

- PEMDAS
- comparison
- boolean operations (and/or/not)

TYPEERROR

Some operands cannot be used between some types

Consider:

```
x = 3 + "Howdy"
y = "bleh" > True
```

Most of the time you can resolve this by just adding a type conversion

INVALID CONVERSIONS

Not all types can always be converted from one to another.
Consider:

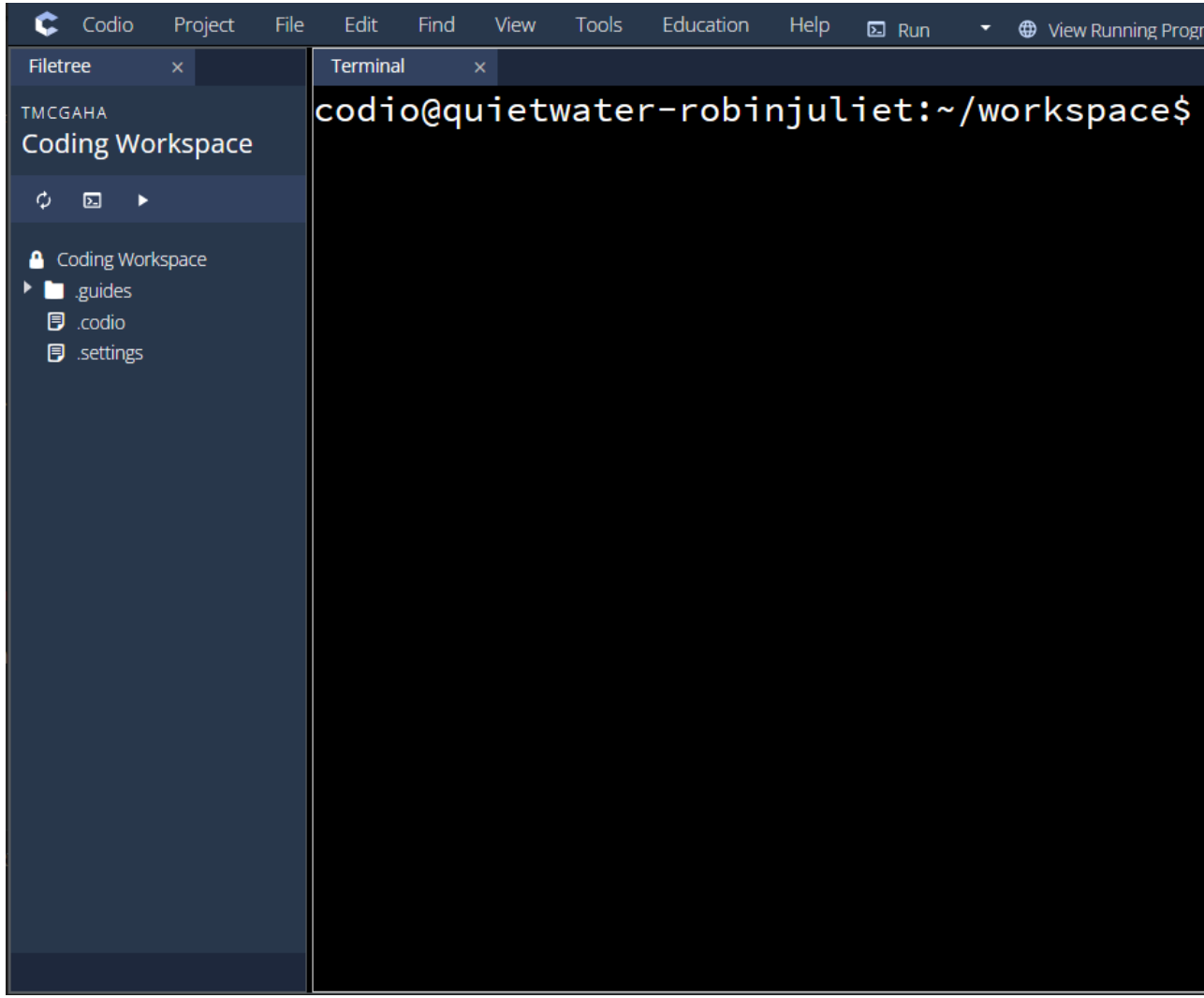
```
x = int("Howdy") # invalid literal for int() with base 10: 'howdy'
```

PYTHON REPL

Python has a REPL that you can use to run python code without storing it in a file.

REPL = **R**ead **E**valuate **P**rint **L**oop

Useful if you want to just check something real quick

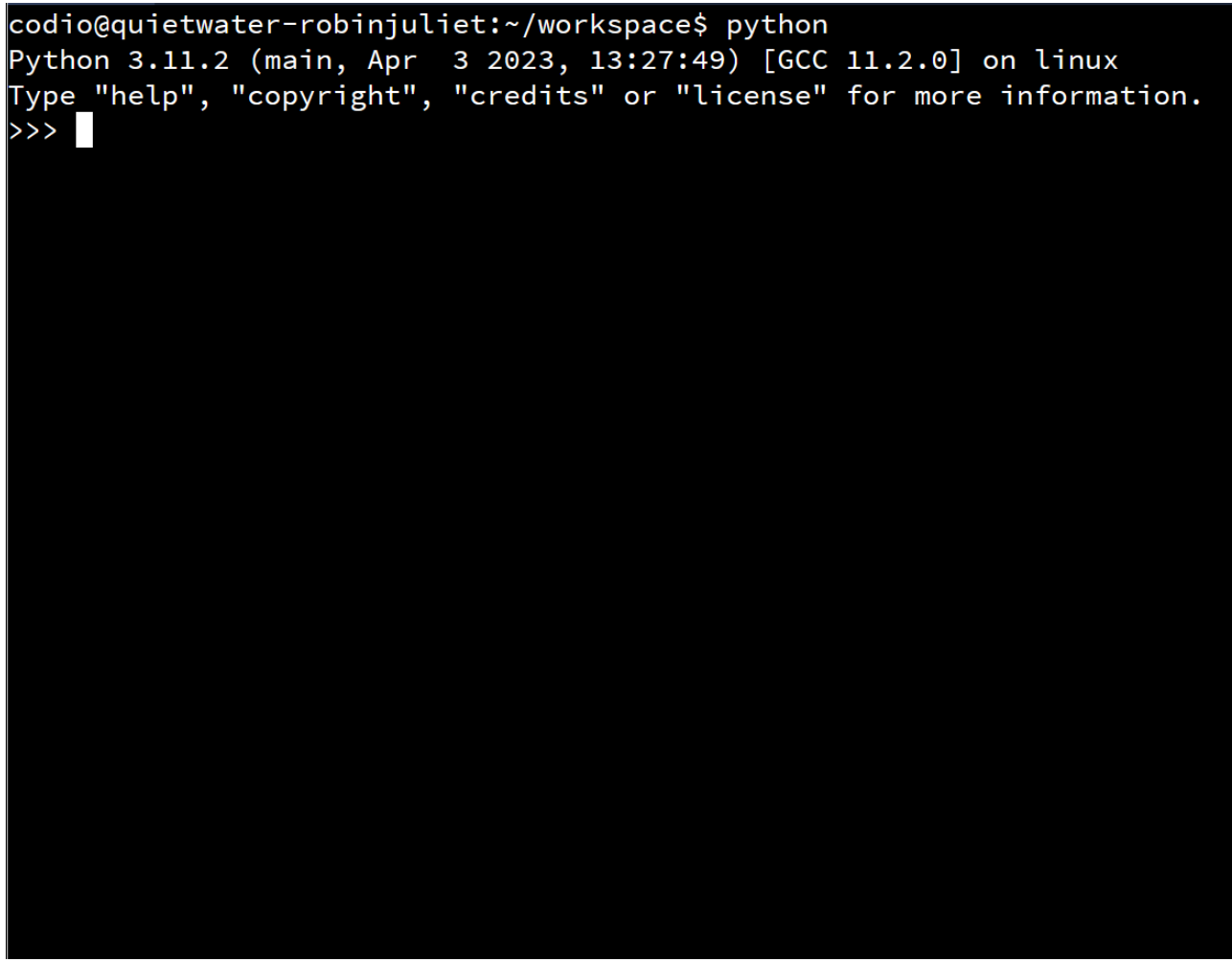


PYTHON REPL

To run, you can go to the "Run" or "terminal" tab in codio and type the command `python`

Remember to end any running program first before you try to run the REPL

Once you have it open, you should be able to see it prompt you in the terminal:



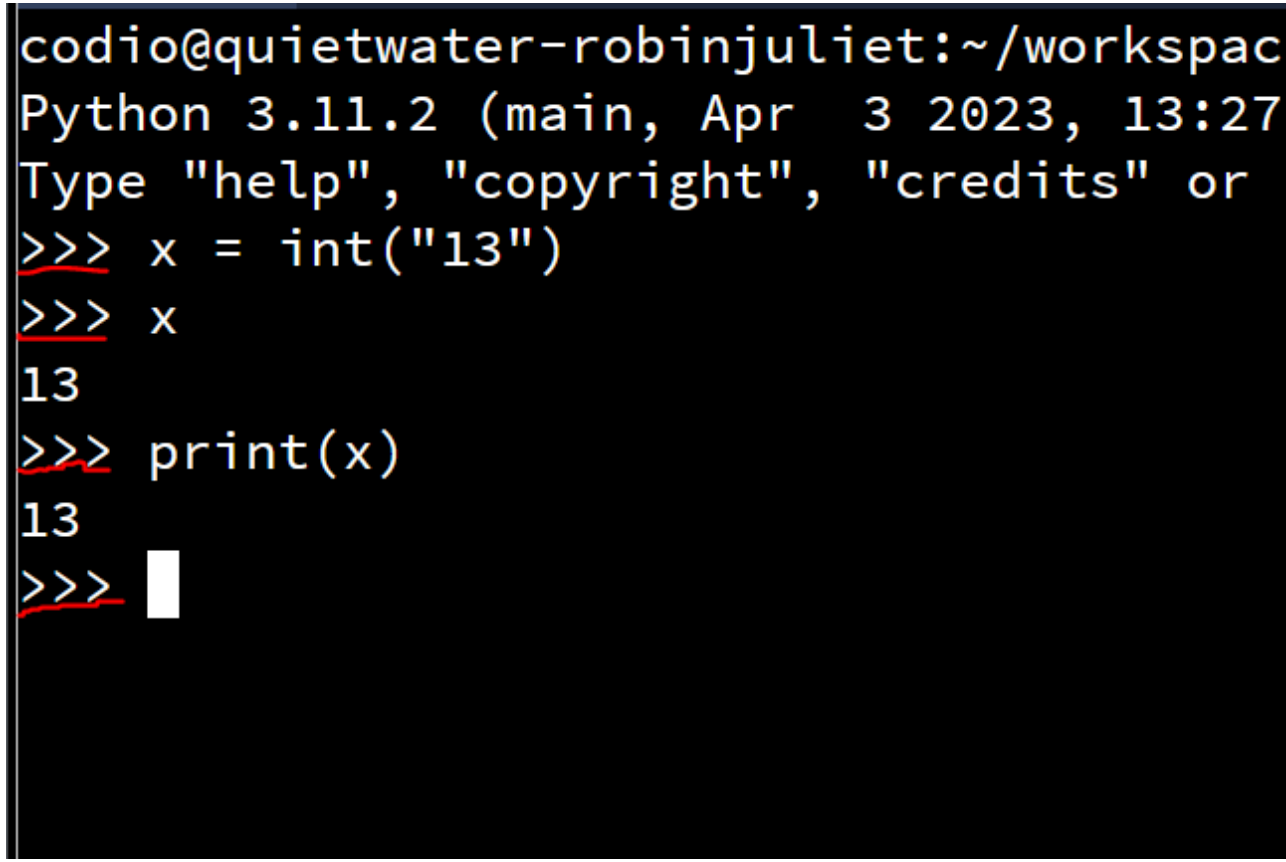
PYTHON REPL

Once you have it open, you should be able to see it prompt you in the terminal: `>>>`

You can then type in lines of python and it will run those lines and remember variables.

Typing just an expression shows its value.

Use `CTRL + D` or type `exit()` when done.



SEQUENCE TYPES: STRING

Strings are **sequences**! Specifically, strings are sequences of characters.

- Sequences have a length
- Sequences have indices for individual members of the sequences

The order of the characters matters!

```
s1 = "hi"
s2 = "ih"
print(s1 == s2) # False
```

SEQUENCE TYPES: STRING

Ordering is an important aspect to strings, and we use **indices** as a way to specify positions in the string.

Consider the string

INDEX	0	1	2	3	4	5
characters	H	e	l	l	o	!

index `0` is the first position (first character)

`len(string) - 1` is the index of the last character

SEQUENCE TYPES: BASIC FUNCTIONALITY

For any sequence, you can use

- `len(seq)` to find the length of the sequence `seq`
- `seq[i]` to access the `i`th index of the sequence `seq`

```
x = "Panda Bear"
print(x[0])           # 'P'
print(x[3])           # 'd'
print(x[len(x) - 1]) # 'r'
```

PRACTICE (L11 FOR BOTH)

What is the index of the letter `E` and `Y` from the following string

```
x = "GY! BE"
```

Get the last character of a string. Your code should work for any value of string that has len >= 1. Do not just say `last_char = "e"`.

```
string = "example"
last_char = ________
```

PRACTICE: C12

Get the middle character of a string withouth knowing the string's value. Assume length is odd and >= 1. Hint: `//`

```
string = "example"
middle_char = _______
```

Get the age from this string an int. Do not just say `age = 26`, actually extract it from the string.

```
string = "Age: 26"
age = ___________________
```

Hint: assume age is two characters. Think about what operators are useful here.

OTHER STRING SEQUENCE FUNCTION

- `string.find(target)`: finds the first index that has the target string, or -1 if not found.

example:

```
"Hello".find("l")  # 2
"Phl".find("U")    # -1
"Attack".find("ta") # 2
```